LoRa-Based Wireless Control Link for a Simulated Drone Using ESP32 and SX1278 at 433 MHz

V Pranav Kumar
School of Electronics Engineering (SENSE)
Vellore Institute of Technology, Chennai, India
Register No. 24BEC1263
Digital Communication Systems (BECE306P) · Slot L9 + L10 · Faculty: Dr. B. Nagajayanthi

***Abstract*—This paper presents the design, firmware implementation and bench evaluation of a one-way digital wireless control link that carries three flight-control commands (aileron, elevator and rudder) from a ground controller to a simulated drone using LoRa chirp-spread-spectrum modulation at 433 MHz. Two ESP32 DevKit V1 boards are interfaced with SX1278 (Ai-Thinker RA-02) transceivers over SPI. The transmitter samples a two-axis joystick and a rudder potentiometer with the 12-bit ADC, maps them to 0–180° commands, and frames them with a 16-bit sequence number and a CRC-16/CCITT checksum into an 11-byte packet sent every 50 ms at spreading factor 7, 125 kHz bandwidth and coding rate 4/5 (calculated time on air 41.2 ms). The receiver validates each packet in seven stages before updating three servo PWM outputs and displays RSSI, SNR, packet counts and link status on an OLED. Seven paired readings under two signal conditions show that commands arrived within 1° of the transmitted values, and that the link remained up at an RSSI of about −107 dBm with SNR as low as −6.2 dB, inside the SF7 demodulation limit of about −7.5 dB. Packet loss was 20.0 % in the strong-signal condition and 20.6–20.9 % in the weak condition. Because the loss barely changed across a ~35 dB difference in RSSI, no acknowledgements were received, and acknowledgements are requested on every fifth packet, the loss is attributed mainly to the acknowledgement exchange rather than to the radio channel; a verification test is proposed.**

***Index Terms*—LoRa, chirp spread spectrum, SX1278, ESP32, packet framing, CRC-16, sequence numbers, RSSI, SNR, failsafe, UAV control link.**

I. Introduction

An unmanned aircraft is steered by a stream of short, low-rate commands: a few control-surface positions refreshed tens of times per second. The radio link that carries them needs reliability and range more than data rate. Low-power wide-area network (LPWAN) technologies were designed for exactly this trade-off [1], [2], and among them LoRa is attractive because its chirp spread spectrum (CSS) modulation exchanges bit rate for processing gain, so packets can be demodulated even when the received signal lies below the noise floor [3], [4].

This work builds such a link from first principles at the packet level, so that the core topics of a Digital Communication Systems course can be observed on real hardware: modulation, digital framing, error detection, sequence-based loss counting and link-quality metrics [5], [6]. The flight controller is represented by a "simulated drone": an ESP32 that drives three servos standing in for the aileron, elevator and rudder.

The objectives of the project are:

1) acquire three analogue control inputs and convert them into calibrated 0–180° commands with 90° as the neutral position;
2) frame the commands into a fixed binary packet with identifier, version, sequence number and CRC-16 checksum;
3) transmit the packets every ~50 ms through an SX1278 transceiver at 433 MHz using LoRa modulation;
4) receive and validate every packet, apply only valid commands to the servo outputs, and reject everything else;
5) present live commands and link statistics (RSSI, SNR, packet counts, link status) on OLED displays at both ends;
6) return all servos to neutral if no valid packet arrives for 500 ms (failsafe); and
7) measure link quality and packet loss under two signal conditions and interpret the results.

The rest of the paper is organised as follows. Section II reviews related work. Section III summarises the LoRa physical-layer theory used to choose the radio parameters. Sections IV and V describe the architecture and the hardware with full circuit diagrams. Sections VI and VII describe the packet format and firmware. Sections VIII–X present the experimental method, the results and their analysis. Sections XI–XIII cover limitations, future work and conclusions.

II. Related Work

Raza *et al.* [1] and Centenaro *et al.* [2] survey LPWAN technologies and position LoRa as a long-range, low-rate option operating in unlicensed sub-GHz bands. Augustin *et al.* [7] describe the LoRa physical layer and the LoRaWAN MAC, and report measured range and throughput. The mathematical description of LoRa as frequency-shift chirp modulation, with its orthogonal symbol set and processing gain, is given by Vangelista [4]; Semtech's application notes [3], [8] provide the practical design equations for symbol time, bit rate and time on air used in Section III.

Field measurements by Petäjäjärvi *et al.* [9] demonstrated LoRa ranges of several kilometres and proposed a channel-attenuation model. Studies of capacity and scalability [10–12] show that throughput is ultimately limited by time on air, duty-cycle rules and collisions between transmissions. That last result is directly relevant here: in Section X the measured loss is traced not to the channel but to a timing interaction between command and acknowledgement packets on a half-duplex radio.

Error detection in this work uses a cyclic redundancy check, whose theory dates to Peterson and Brown [13]. The hardware is built on the ESP32 system-on-chip [14] and the SX1278 transceiver [15], driven through the open-source arduino-LoRa library [16].

III. LoRa Physical Layer and Link Design

A. Chirp Spread Spectrum

A LoRa symbol is a linear frequency chirp that sweeps the full bandwidth BW. The spreading factor SF sets the number of chips per symbol, 2^{SF}, and each symbol carries SF bits encoded as the cyclic starting frequency of the chirp [4]. The symbol duration is

$$T_s = 2^{SF} / BW \tag{1}$$

and the useful bit rate after forward error correction at code rate $CR = 4/(4 + n)$ is [3]

$$R_b = SF \cdot (BW / 2^{SF}) \cdot 4/(4 + n). \tag{2}$$

For SF7, BW = 125 kHz and CR 4/5, T_s = 1.024 ms and $R_b \approx$ 5.47 kbit/s. Each increase of SF by one doubles T_s and adds about 2.5 dB of demodulation margin, so the demodulator can operate down to SNR ≈ −7.5 dB at SF7 and ≈ −20 dB at SF12 [15].

B. Time on Air

With an explicit header, payload CRC enabled and low-data-rate optimisation off (DE = 0), the number of payload symbols for a payload of PL bytes is [8], with CR = 1 for code rate 4/5,

$$n_{pl} = 8 + \lceil (8PL - 4SF + 44) / 4SF \rceil \cdot (CR + 4) \tag{3}$$

and the time on air with an n_{pre}-symbol preamble is

$$T_{air} = (n_{pre} + 4.25 + n_{pl}) \cdot T_s. \tag{4}$$

For the 11-byte command packet, $n_{pl} = 8 + 4{\cdot}5 = 28$ symbols and $T_{air} = (8 + 4.25 + 28) \times 1.024$ ms = 41.2 ms. Fig. 1 applies (3)–(4) for every spreading factor. Only SF7 fits within the 50 ms command interval, which is why SF7 was chosen; SF8 already needs 82.4 ms.

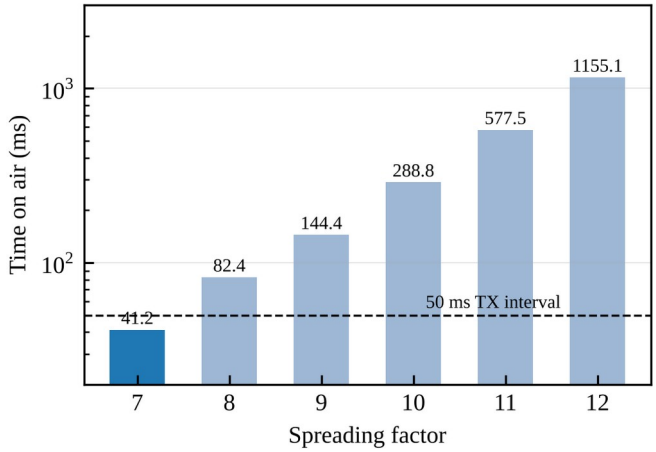


Fig. 1. Calculated time on air of the 11-byte command packet versus spreading factor (125 kHz, CR 4/5, 8-symbol preamble, explicit header, CRC on). The dashed line marks the 50 ms command interval.

C. Processing Gain and Sensitivity

The CSS processing gain at SF7 is $10 \log_{10}(2^7) \approx 21$ dB. Receiver sensitivity is

$$S = -174 + 10 \log_{10} BW + NF + \mathrm{SNR}_{lim} \ \text{(dBm)} \tag{5}$$

which, with a 6 dB noise figure, BW = 125 kHz and SNR_{lim} = −7.5 dB, gives about −124 dBm, consistent with the −123 dBm datasheet figure for SF7 [15]. Near sensitivity the RSSI reading includes the noise power, so SNR is the more meaningful margin metric.

D. Radio Configuration

Table I lists the parameters, which are identical in both sketches through a shared definition block. A mismatch in any of them prevents communication entirely.

TABLE I
Radio Configuration (Both Nodes)

Parameter	Value	Firmware constant
Carrier frequency	433.000 MHz	`LORA_FREQUENCY_HZ`
Bandwidth	125 kHz	`LORA_BANDWIDTH_HZ`
Spreading factor	SF7 (128 chips/symbol)	`LORA_SPREADING_FACTOR`
Coding rate	4/5	`LORA_CODING_RATE_DENOM=5`
Sync word	0x24 (private)	`LORA_SYNC_WORD`
Preamble	8 symbols	`LORA_PREAMBLE_SYMBOLS`
TX power	10 dBm, PA_BOOST	`LORA_TX_POWER_DBM`
Header / CRC	Explicit, HW CRC on	`LoRa.enableCrc()`
SPI clock	1 MHz	`LORA_SPI_FREQUENCY_HZ`
Interval / timeout	50 ms / 500 ms	`TX_INTERVAL_MS`, `LINK_TIMEOUT_MS`

IV. System Architecture

The system has two independent ESP32 nodes sharing one 433 MHz LoRa channel (Fig. 2). The transmitter turns stick positions into framed digital packets; the receiver recovers, checks and applies them. The dashed acknowledgement path is implemented in firmware but produced no received acknowledgements during testing. Table II separates what has been demonstrated from what is only implemented or proposed; the rest of the paper uses the same categories.

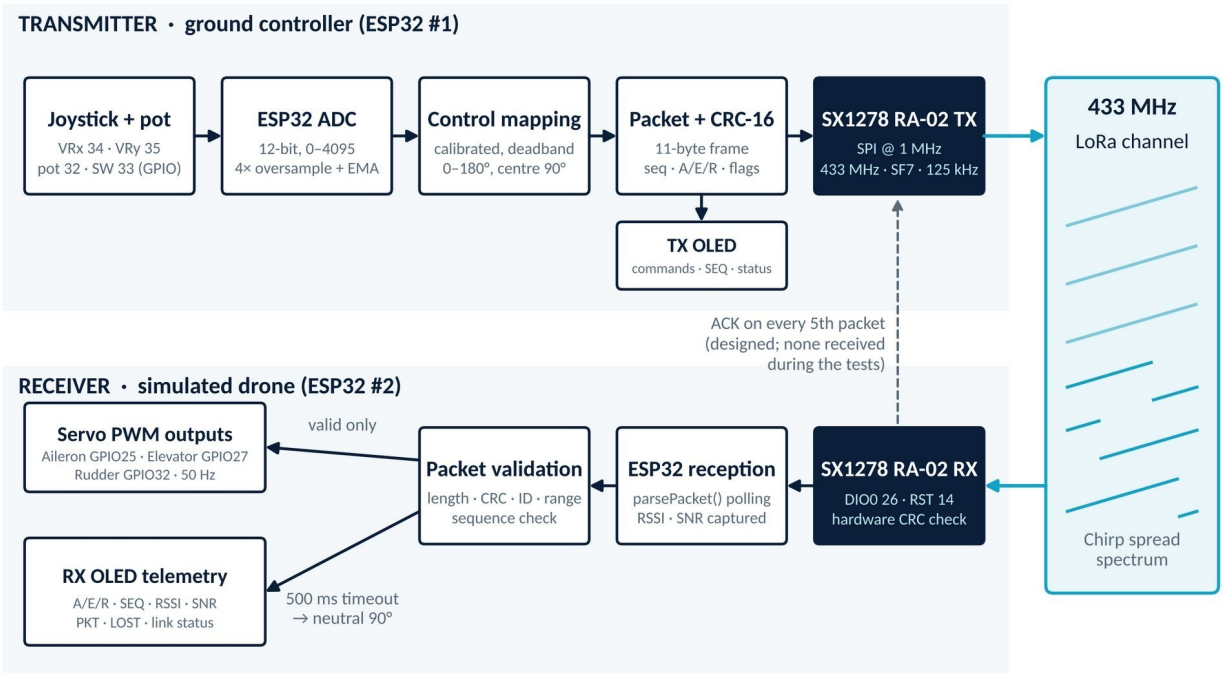


Fig. 2. System architecture. Solid arrows are the forward command path. The dashed arrow is the acknowledgement path, which is implemented in firmware but produced no received acknowledgements during testing.

TABLE II
IMPLEMENTATION AND VERIFICATION STATUS

Function	Status	Evidence
Input acquisition, 0–180° mapping	Working, observed	TX OLED commands follow stick and pot (Fig. 7).
Framing, sequence no., CRC-16	Working, observed	RX values match TX within 1° (Table VIII).
LoRa TX/RX at SF7	Working, observed	RX shows LINK OK at −68 to −108 dBm.
Receiver validation pipeline	Working, observed	No corrupted value displayed; reject counts not logged.
OLED telemetry, both ends	Working, observed	Figs. 7, 8, 18–21.
Servo PWM (GPIO 25, 27, 32)	Implemented, not measured	In firmware; motion not recorded.
500 ms failsafe	Implemented, not measured	In firmware; not exercised.
ACK every 5th packet	Not working	ACK count 0 in every TX reading.
Flight-controller link	Proposed	Future work (Section XII).

TABLE III
COMPONENTS USED

Component	Qty	Role
ESP32 DevKit V1 (ESP32-WROOM-32)	2	Node controller: 12-bit ADC, SPI, I²C, LEDC PWM
SX1278 Ai-Thinker RA-02 (433 MHz) + antenna	2	LoRa transceiver, SPI slave, PA_BOOST
0.96″ SSD1306 OLED, 128×64, I²C 0x3C	2	Local telemetry display
Two-axis analogue joystick with switch	1	Aileron (X), elevator (Y), push switch
Linear potentiometer	1	Rudder input (10 kΩ specified; unit used is 1 kΩ)
Micro servo, SG90 class	3	Simulated aileron, elevator, rudder
Buck converter module	2	3.3 V logic rail and separate 5 V servo rail
Battery pack	1	Supply to both converters

V. HARDWARE DESIGN AND CIRCUIT DIAGRAMS

Both nodes are built on perforated boards around an ESP32 DevKit V1 [14] and an RA-02 module [15]. Table III lists the components. Figs. 3–5 are the circuit diagrams drawn from the final, firmware-verified pin assignments, and Tables IV and V list them per board because the two boards do not share a pinout.

A. Transmitter Circuit

In the transmitter (Fig. 3) the RA-02 is connected to the ESP32 VSPI bus (SCK 18, MISO 19, MOSI 23) with NSS on GPIO 5, RESET on GPIO 14 and DIO0 on GPIO 26. The joystick axes and the rudder potentiometer wiper go to ADC1 channels on GPIO 34, 35 and 32; ADC1 is used because ADC2 is unavailable while the radio stack is active. The joystick switch uses GPIO 33 with the internal pull-up (active low, 30 ms debounce). The OLED shares the I²C bus on GPIO 21/22.

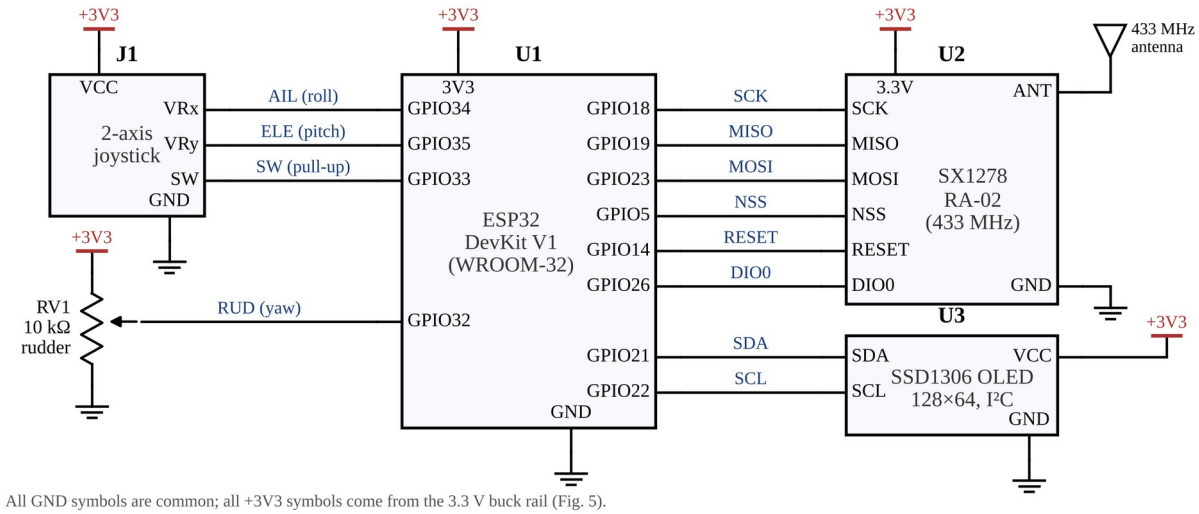


Fig. 3. Circuit diagram of the transmitter (ground controller). U1 ESP32 DevKit V1, U2 SX1278 RA-02, U3 SSD1306 OLED, J1 joystick, RV1 rudder potentiometer.

TABLE IV
TRANSMITTER (ESP32 #1) INTERFACING

Device	Signal	ESP32 pin	Function
SX1278 RA-02	SCK/MISO/MOSI	GPIO 18/19/23	SPI (VSPI) register/FIFO access
	NSS · RST · DIO0	GPIO 5 · 14 · 26	Chip select, reset, IRQ (polled)
Joystick	VRx	GPIO 34 (ADC1_CH6)	Aileron (roll)
	VRy	GPIO 35 (ADC1_CH7)	Elevator (pitch)
	SW	GPIO 33, pull-up	Push button, active low
Potentiometer	Wiper	GPIO 32 (ADC1_CH4)	Rudder (yaw)
SSD1306 OLED	SDA/SCL	GPIO 21/22	Commands, SEQ, TX/ACK counts

B. Receiver Circuit

The receiver (Fig. 4) uses the same radio and display wiring. The three servo signal lines are driven by LEDC PWM channels at 50 Hz on GPIO 25, 27 and 32 with 1000–2000 µs pulses. GPIO 26 is reserved for DIO0 only, which corrects the design-stage drawing of Fig. 6, where the elevator servo was shown on GPIO 26.

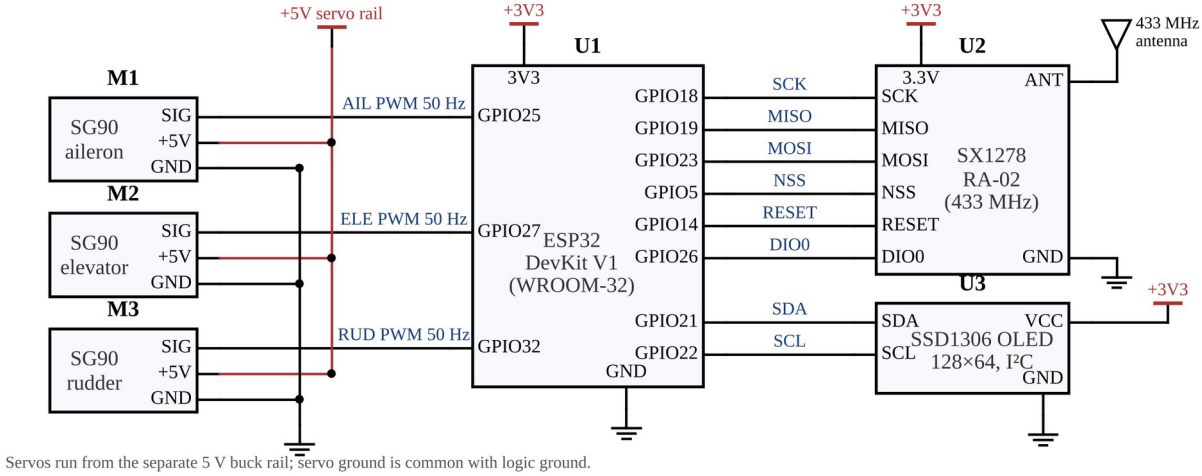


Fig. 4. Circuit diagram of the receiver (simulated drone). M1–M3 are SG90 servos on a separate 5 V rail with common ground.

TABLE V
RECEIVER (ESP32 #2) INTERFACING

Device	Signal	ESP32 pin	Function
SX1278 RA-02	SCK/MISO/MOSI	GPIO 18/19/23	SPI (VSPI)
	NSS · RST · DIO0	GPIO 5 · 14 · 26	GPIO 26 reserved for DIO0
Aileron servo	Signal	GPIO 25	LEDC PWM 50 Hz, 1–2 ms
Elevator servo	Signal	GPIO 27	LEDC PWM 50 Hz, 1–2 ms
Rudder servo	Signal	GPIO 32	LEDC PWM 50 Hz, 1–2 ms
SSD1306 OLED	SDA/SCL	GPIO 21/22	A/E/R, SEQ, RSSI, SNR, PKT, LOST

C. Power Distribution

The RA-02 and the OLEDs are 3.3 V devices and are not 5 V tolerant. Servo current peaks far exceed what the ESP32 regulator can supply, so the servos run from a separate 5 V buck converter with a bulk capacitor, while a second converter supplies the 3.3 V logic rail (Fig. 5). All grounds are common.

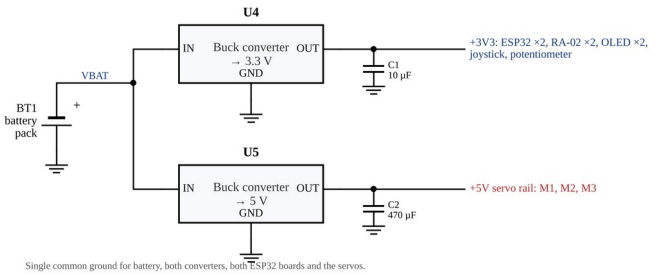


Fig. 5. Power-distribution circuit: one battery feeds a 3.3 V logic converter and a separate 5 V servo converter with a common ground.

Fig. 6 is the wiring diagram prepared at the design stage. It is kept for completeness, but its servo labels and 5-byte packet box predate the final firmware; Figs. 3–5 and Tables IV–VI are authoritative. Figs. 7–9 show the built hardware.

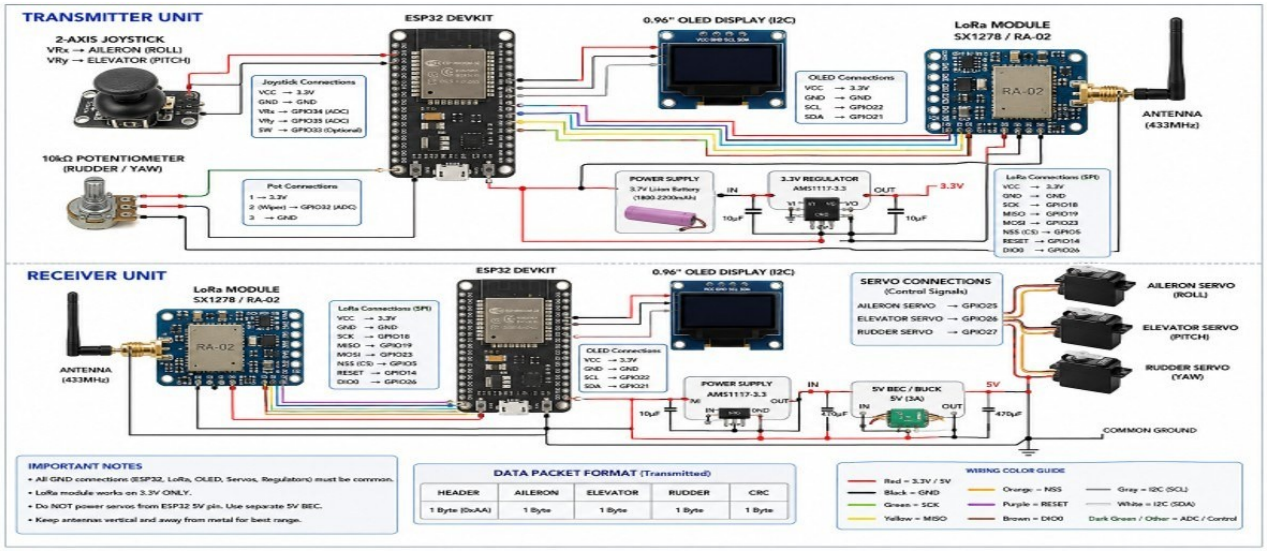
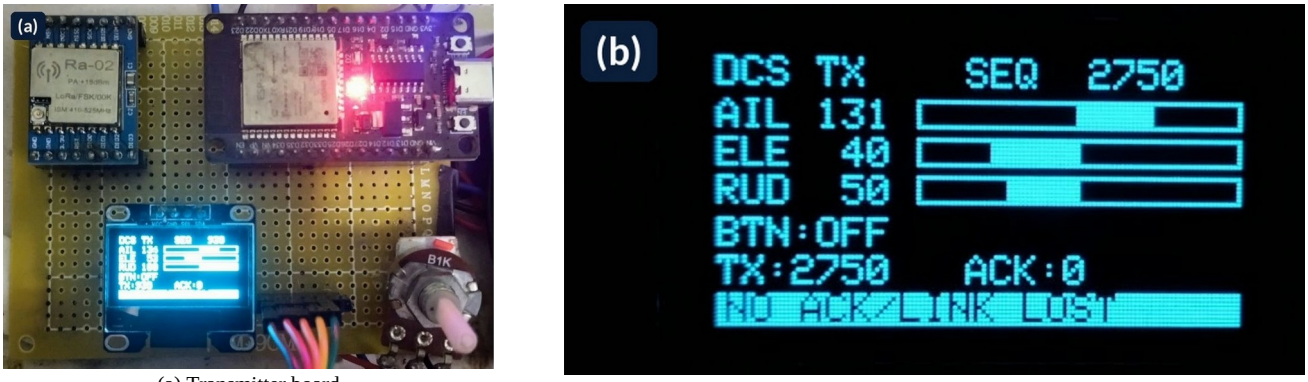


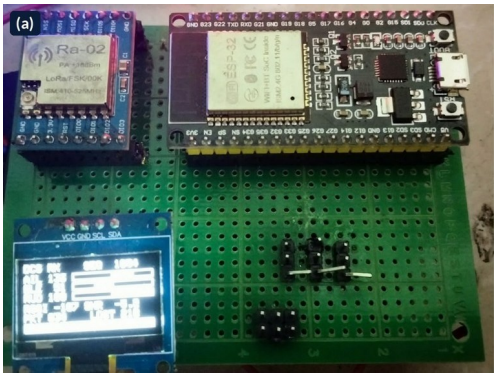
Fig. 6. Design-stage wiring diagram. Its servo labels (elevator on GPIO 26, rudder on GPIO 27) and 5-byte packet box are superseded by the final firmware.



(a) Transmitter board

(b) Transmitter OLED, Observation 4

Fig. 7. Transmitter node. (a) ESP32, RA-02, OLED and rudder potentiometer on perfboard. (b) OLED showing commands with centre-referenced bars, sequence number, packet and ACK counts, and status.



(a) Receiver board



(b) Receiver OLED, Observation 4

Fig. 8. Receiver node. (a) ESP32, RA-02, OLED and three-way servo header. (b) OLED showing applied commands, RSSI −107 dBm, SNR −6.2 dB, 2248 packets received, 594 lost, LINK OK.

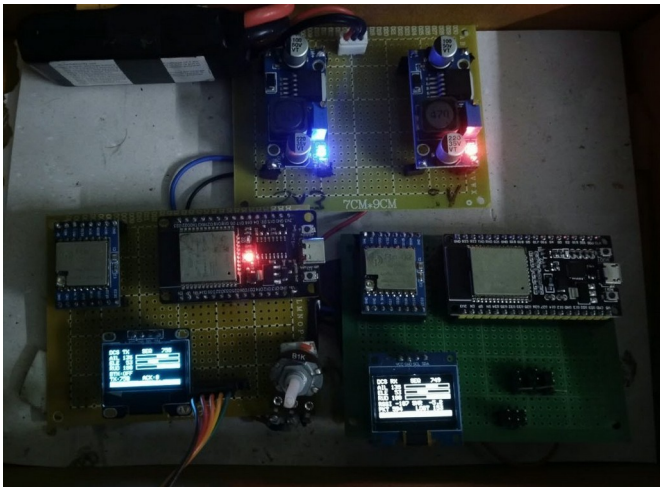


Fig. 9. Complete bench setup: battery and the two buck converters (top), transmitter node (left) and receiver node (right).

VI. Packet Structure and Error Detection

The command packet is an 11-byte little-endian binary frame (Table VI), built by `buildCommandPacket()` in Transmitter.ino and parsed by `decodeCommandPacket()` in Receiver.ino. The CRC covers bytes 0–8 and is stored low byte first.

TABLE VI
Command Packet Format (11 Bytes)

Byte	Field	Value	Purpose
0	Identifier	0xDC (`PROTO_MAGIC`)	Rejects foreign LoRa traffic
1	Version	0x01	Rejects incompatible firmware
2	Type	0x01 = COMMAND	Separates commands from ACKs (0x02)
3–4	Sequence no.	uint16, wraps at 65535	Loss, duplicate, order detection
5	Aileron	0–180°	Roll command
6	Elevator	0–180°	Pitch command
7	Rudder	0–180°	Yaw command
8	Flags	b0 button, b1 ACK req.	Button state, ACK request
9–10	CRC-16	CCITT-FALSE, bytes 0–8	Application-level integrity

The CRC treats the 72-bit message $M(x)$ as a polynomial over GF(2) and appends the remainder [13]

$$R(x) = x^{16}\,M(x) \bmod G(x), \quad G(x) = x^{16} + x^{12} + x^5 + 1 \qquad (6)$$

with initial value 0xFFFF. This detects all single-, double- and odd-weight bit errors and all bursts up to 16 bits [5]. It is applied on top of the SX1278 hardware payload CRC, so a frame must pass both.

When the ACK-request flag is set, the receiver is designed to reply with a 12-byte acknowledgement carrying the same header with type 0x02, the acknowledged sequence number, a status byte, the command RSSI (int8, dBm) and SNR (int8, ×4), the receiver's 16-bit valid-packet counter and a CRC-16 over bytes 0–9.

A command is applied only if it passes every stage in Fig. 10. The sequence check uses 16-bit modulo arithmetic: a difference of 0 is a duplicate, 1 to 32 767 is newer (any gap is added to the lost counter), and anything else is old. After LINK LOST, or after 10 consecutive "old" packets, the receiver resynchronises to the incoming sequence number.

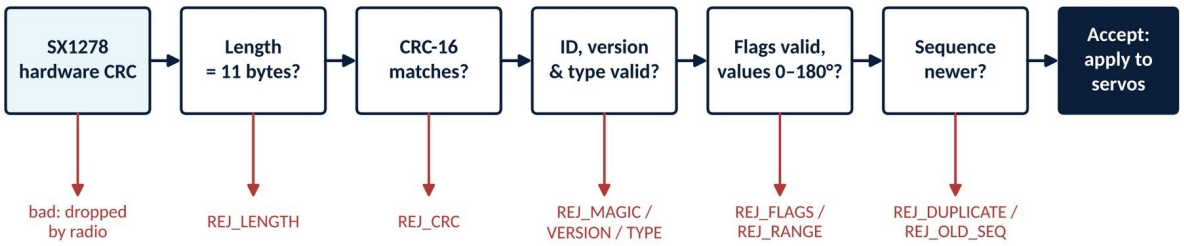


Fig. 10. Receiver acceptance pipeline. The hardware CRC stage runs inside the SX1278 and the LoRa library; the remaining stages run in `decodeCommandPacket()` and `checkSequence()`. Every rejection increments the invalid counter and is logged on Serial with its reason.

VII. Firmware Implementation

Both sketches are written for the Arduino-ESP32 core and use the arduino-LoRa library [16], Adafruit SSD1306/GFX and, on the receiver, ESP32Servo. Neither `loop()` uses long blocking delays. The transmitter runs a three-state radio state machine (IDLE → SENDING → ACK_LISTEN) and schedules OLED refreshes only when they cannot delay a transmission. The receiver polls the radio on every pass, checks the 500 ms timeout, and redraws the display only after a packet or when the channel is quiet. Figs. 11–17 show short excerpts copied verbatim from the two sketches; line numbers are local to each excerpt.

A. Transmitter

In Fig. 11, lines 1–3 place the aileron, elevator and rudder commands into bytes 5–7, each passing through `clampDegrees()` so no value outside 0–180° can be framed even if calibration is wrong. Line 4 masks the flags to the two defined bits. Line 5 computes the CRC over the first nine bytes, so the checksum protects every field including the sequence number; line 6 stores its low byte and the next line the high byte.

In Fig. 12, `beginPacket()` has already confirmed that the radio finished the previous packet (otherwise the slot is retried 2 ms later). Line 1 takes and post-increments the sequence number. Lines 2–3 build the frame in a fixed 11-byte stack buffer. Line 5 copies it over SPI into the SX1278 FIFO, and line 6 starts an asynchronous transmission: the SX1278 autonomously sends preamble, header, payload and hardware CRC as CSS symbols. `packetsSent`, the TX count on the OLED, therefore means “transmitted”, not “delivered”.

Transmitter.ino excerpt · buildCommandPacket()

```
1 f[5] = clampDegrees(deg[AXIS_AILERON]);
2 f[6] = clampDegrees(deg[AXIS_ELEVATOR]);
3 f[7] = clampDegrees(deg[AXIS_RUDDER]);
4 f[8] = (uint8_t)(flags & CMD_FLAGS_DEFINED_MASK);
5 const uint16_t crc = crc16Ccitt(f, CMD_PACKET_LEN - 2);
6 f[9]  = (uint8_t)(crc & 0xFF);
```

Fig. 11. Transmitter.ino excerpt: packet construction in `buildCommandPacket()`.

Transmitter.ino excerpt · startCommandTransmission()

```
1 const uint16_t seq = nextSeq++;
2 uint8_t frame[CMD_PACKET_LEN];
3 buildCommandPacket(frame, seq, controls.commandDeg, flags);
4
5 LoRa.write(frame, CMD_PACKET_LEN);
6 LoRa.endPacket(true);   // async: returns immediately; SX1278 transmits on its own
```

Fig. 12. Transmitter.ino excerpt: handing the packet to the radio in `startCommandTransmission()`.

B. Receiver

`LoRa.parsePacket()` is called on every pass; it returns 0 when nothing has arrived, or the payload length of a packet that passed the hardware CRC, and re-arms single-shot receive mode. In Fig. 13, line 1 drains the FIFO into a 32-byte buffer, discarding any excess so an oversized packet cannot overflow memory; lines 2–3 read the packet RSSI and SNR; line 7 runs the structural checks and line 8 the sequence check.

In Fig. 14, the length test compares both the reported and stored lengths with 11 bytes; lines 2–3 reassemble the received CRC and compare it with one recomputed over bytes 0–8, so a single flipped bit fails here; lines 4–6 confirm identifier, version and type. In Fig. 15, the 16-bit modulo difference handles wrap-around from 65535 to 0, a zero difference is rejected as a duplicate, and a forward step larger than one adds the gap to `lostCount`, the LOST value on the OLED. LOST therefore counts sequence gaps: packets that never arrived and packets that arrived but were rejected.

Receiver.ino excerpt · handleIncomingPacket()

```
const int stored = readPacketBytes(buf, MAX_LORA_PAYLOAD);
const int rssi = LoRa.packetRssi();
const float snr = LoRa.packetSnr();
lastPacketActivityMs = now;

CommandPacket cmd;
RejectReason reason = decodeCommandPacket(buf, stored, reportedLen, cmd);
if (reason == REJ_NONE) reason = checkSequence(cmd.seq);
```

Fig. 13. Receiver.ino excerpt: packet reception in `handleIncomingPacket()`.

Receiver.ino excerpt · decodeCommandPacket()

```
if (reportedLen != CMD_PACKET_LEN || storedLen != CMD_PACKET_LEN) return REJ_LENGTH;
const uint16_t rxCrc = (uint16_t)b[9] | ((uint16_t)b[10] << 8);
if (crc16Ccitt(b, CMD_PACKET_LEN - 2) != rxCrc) return REJ_CRC;
if (b[0] != PROTO_MAGIC)      return REJ_MAGIC;
if (b[1] != PROTO_VERSION)    return REJ_VERSION;
if (b[2] != PKT_TYPE_COMMAND) return REJ_TYPE;
```

Fig. 14. Receiver.ino excerpt: structural validation in `decodeCommandPacket()`.

Receiver.ino excerpt · checkSequence()

```
const uint16_t diff = (uint16_t)(seq - lastSeq);
if (diff == 0) {
  duplicateCount++;
  return REJ_DUPLICATE;
}
if (diff < 0x8000) {
  if (diff > 1) {
    lostCount += (uint32_t)(diff - 1);
```

Fig. 15. Receiver.ino excerpt: sequence check in `checkSequence()`.

C. Servo Output and OLED Telemetry

After a packet is accepted, `writeServo()` (Fig. 16) maps 0–180° linearly onto a 1000–2000 µs pulse in integer arithmetic (the +90 term rounds to the nearest microsecond), updates the LEDC duty cycle only if the channel attached at start-up, and records the applied angle, which is what the OLED displays.

Both OLEDs share a layout: a header with node and sequence number, three A/E/R rows with bars drawn outward from the 90° mark, two statistics lines and an inverted status bar. Fig. 17 prints RSSI and SNR only after the first valid packet and writes the link state. Table VII lists the status messages.

Receiver.ino excerpt · writeServo()

```
const long us = SERVO_PULSE_MIN_US +
    ((long)(SERVO_PULSE_MAX_US - SERVO_PULSE_MIN_US) * deg + 90) / 180;
if (servos[i].attached()) servos[i].writeMicroseconds((int)us);
appliedDeg[i] = deg;
```

Fig. 16. Receiver.ino excerpt: servo pulse mapping in `writeServo()`.

Receiver.ino excerpt · drawDisplay()

```
if (validCount > 0) snprintf(line, sizeof(line), "RSSI %4d SNR %5.1f", lastRssi, lastSnr);
else                snprintf(line, sizeof(line), "RSSI ---- SNR ---");
display.setCursor(0, 38);
display.print(line);
// ...
const RxLinkStatus st = currentLinkStatus(now);
display.fillRect(0, 56, OLED_WIDTH, 8, SSD1306_WHITE);
display.setTextColor(SSD1306_BLACK);
display.setCursor(2, 56);
display.print(linkStatusText(st));
```

Fig. 17. Receiver.ino excerpt: link statistics and status bar in `drawDisplay()`.

TABLE VII
OLED STATUS MESSAGES

Node	Status	Condition in firmware
TX	RADIO READY	Radio initialised, nothing sent yet
TX	TRANSMITTING	Sending; no ACK confirmed yet (first second)
TX	ACK RECEIVED	Valid ACK within the last 1000 ms
TX	NO ACK/LINK LOST	No valid ACK for >1000 ms; reflects the return path only
RX	WAITING	No valid command since start-up
RX	LINK OK	Valid command within the last 500 ms
RX	LINK LOST	Timeout exceeded; servos set to neutral
Both	LORA INIT FAIL	SX1278 not detected over SPI; retry every 2 s

This distinction matters when reading the results: during every test the transmitter displayed NO ACK/LINK LOST while the receiver displayed LINK OK. The forward command link worked throughout; only the acknowledgement return path failed.

VIII. Experimental Setup and Methodology

- **Configuration.** Both nodes ran the firmware of Section VII with the parameters of Table I, acknowledgements enabled (`ENABLE_ACK = true`, one request every 5 packets) and a 50 ms command interval, powered as in Fig. 5.
- **Data capture.** Readings were taken by photographing both OLEDs in the same frame (Fig. 18), so each pair shows transmitter and receiver at the same instant to within one packet interval.
- **Signal conditions.** Observations 1–4 form Set A (weak, RSSI ≈ −107 dBm); Observations 5–7 form Set B (strong, −68 to −77 dBm). The sets are defined by measured RSSI; the physical arrangement that produced each condition was not recorded.
- **Observation 4.** Its TX and RX values come from two separate close-up photographs (Figs. 7(b) and 8(b)); its RX SEQ (2844) exceeds its TX SEQ (2750), so it is not a simultaneous pair and is marked †.
- **Counters.** PKT and LOST are cumulative from receiver start-up. Packet loss is computed with (7).

$$P_{\text{loss}} = N_{\text{lost}} / (N_{\text{rcvd}} + N_{\text{lost}}) \times 100\ \% \qquad (7)$$

IX. Results

Table VIII reproduces the recorded observations without modification, and Table IX summarises them by signal condition. Fig. 18 shows the raw OLED photographs and Figs. 20 and 21 enlarge each display pair so the values in Table VIII can be read directly.

A video demonstration of the working prototype is available at https://drive.google.com/drive/folders/17KGEKCCFoPgjZRtqW9QMCZfphlg0HTpz?usp=sharing.

TABLE VIII
Recorded Observations (Transmitter and Receiver OLEDs)

Obs	Set	TX SEQ	TX A/E/R (°)	RX SEQ	RX A/E/R (°)	RSSI (dBm)	SNR (dB)	Rcvd	Lost	Loss (%)
1	A	1620	133 / 44 / 57	1619	132 / 44 / 57	−107	−5.5	1284	333	20.6
2	A	1825	24 / 0 / 0	1824	24 / 0 / 0	−107	−3.5	1445	377	20.7
3	A	2330	134 / 41 / 137	2329	134 / 41 / 136	−108	−6.2	1841	486	20.9
4 †	A	2750	131 / 40 / 50	2844	131 / 39 / 50	−107	−6.2	2248	594	20.9
5	B	1945	133 / 37 / 71	1944	133 / 37 / 70	−68	9.2	1420	356	20.0
6	B	2165	134 / 37 / 112	2164	134 / 36 / 112	−68	9.8	1596	400	20.0
7	B	2415	77 / 48 / 44	2414	77 / 48 / 44	−77	9.8	1796	450	20.0

† TX and RX values for Observation 4 come from separate photographs taken at different times. In every reading the transmitter showed ACK: 0 and NO ACK/LINK LOST, and the receiver showed LINK OK. Shaded rows are Set B.

TABLE IX
Summary by Signal Condition

Set	RSSI (dBm)	SNR (dB)	Loss (%)	ACKs	RX status
A (weak), 1–4	−108 to −107	−6.2 to −3.5	20.6–20.9	0	LINK OK
B (strong), 5–7	−77 to −68	+9.2 to +9.8	20.0	0	LINK OK



(a) Obs. 1 (Set A) (b) Obs. 2 (Set A) (c) Obs. 3 (Set A)

Fig. 18. Result screenshots, Set A (weak signal). In each frame the transmitter display (cyan) is on the left and the receiver display (white) on the right. Photographs brightened for legibility; no content altered.



(a) Obs. 5 (Set B) (b) Obs. 6 (Set B) (c) Obs. 7 (Set B)

Fig. 19. Result screenshots, Set B (strong signal), same arrangement as Fig. 18.



(a) Obs. 1: TX SEQ 1620 / RX SEQ 1619, RSSI −107, SNR −5.5



(b) Obs. 2: TX SEQ 1825 / RX SEQ 1824, RSSI −107, SNR −3.5



(c) Obs. 3: TX SEQ 2330 / RX SEQ 2329, RSSI −108, SNR −6.2

Fig. 20. Enlarged OLED pairs for Set A, cropped from Fig. 18 (TX left, RX right). Both counters, all three commands and the negative SNR are legible.



(a) Obs. 5: TX SEQ 1945 / RX SEQ 1944, RSSI −68, SNR 9.2



(b) Obs. 6: TX SEQ 2165 / RX SEQ 2164, RSSI −68, SNR 9.8



(c) Obs. 7: TX SEQ 2415 / RX SEQ 2414, RSSI −77, SNR 9.8

Fig. 21. Enlarged OLED pairs for Set B, cropped from Fig. 19. The transmitter shows ACK: 0 in every frame.

Fig. 22 plots RSSI and SNR per observation; Figs. 23 and 24 plot packet loss against RSSI and SNR; Fig. 25 shows the cumulative received and lost counters; Fig. 26 compares transmitted and received commands.

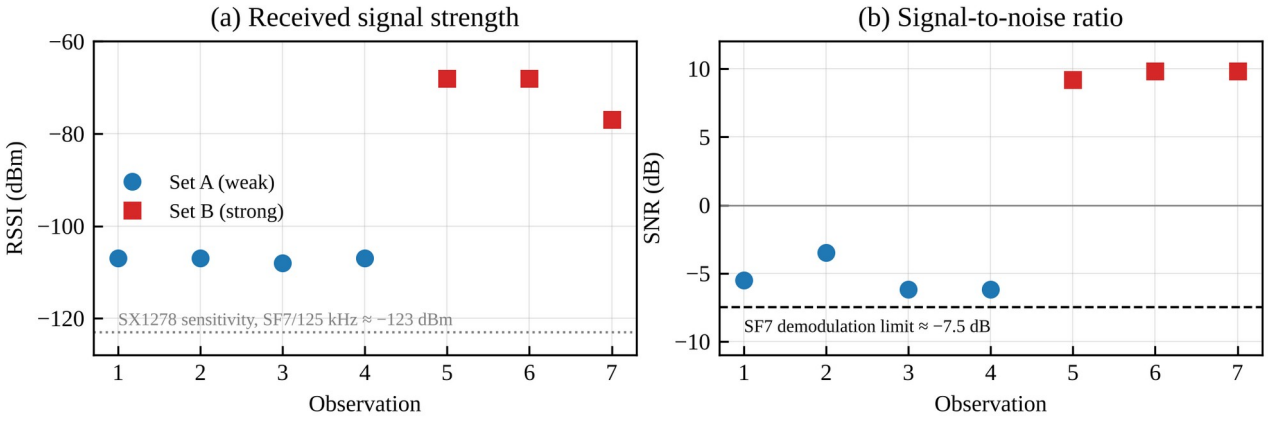


Fig. 22. Measured link metrics per observation. (a) RSSI, with the SF7 datasheet sensitivity. (b) SNR; the dashed line is the SF7 demodulation limit of about −7.5 dB.

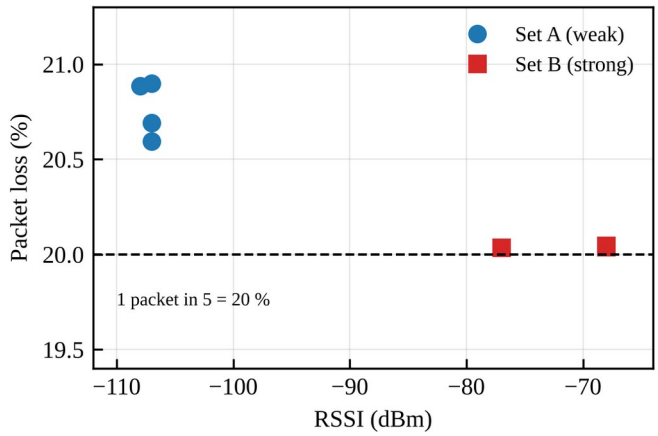


Fig. 23. Packet loss against RSSI. A ~35 dB change in RSSI moves the loss by less than one percentage point; the dashed line marks exactly one packet in five.

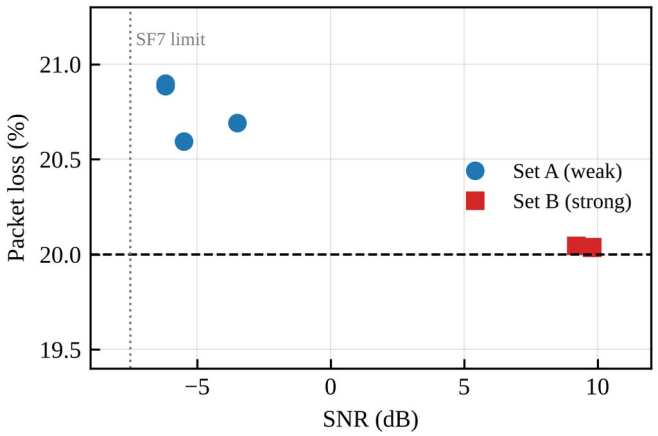


Fig. 24. Packet loss against SNR. Set A operates only 1.3–4.0 dB above the SF7 limit.

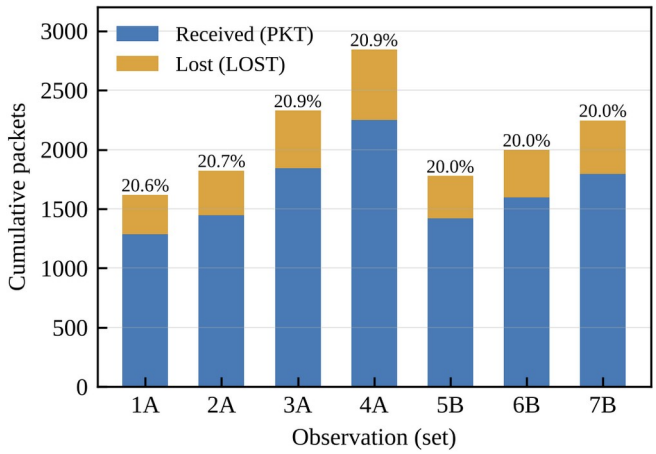


Fig. 25. Cumulative received (PKT) and lost (LOST) packets at each observation, with the resulting loss percentage.

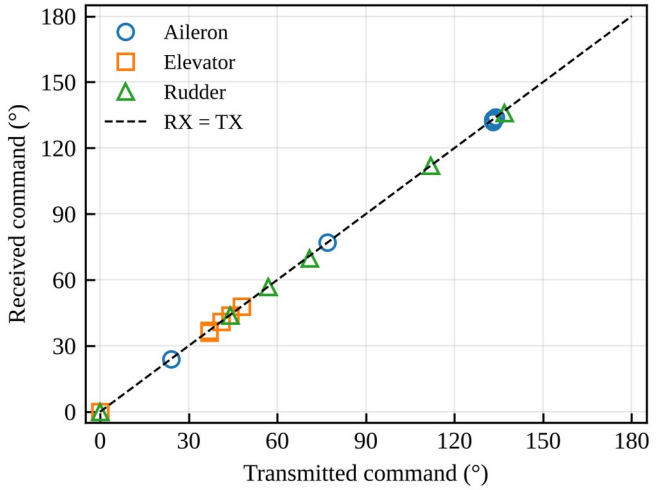


Fig. 26. Received versus transmitted command for the six simultaneous observations (18 values). All points lie on or within 1° of the RX = TX line.

X. Communication Performance Analysis

A. Command Integrity

Across the six simultaneous observations, 14 of the 18 received command values equal the transmitted values exactly and the other 4 differ by exactly 1° (Fig. 26). The differences have a timing explanation, not a transmission error: the receiver always shows packet $n - 1$ while the transmitter shows packet n (RX SEQ = TX SEQ − 1 in every simultaneous row), so a command that changes by one degree between consecutive packets appears 1° apart. No out-of-range or corrupted command was ever displayed, consistent with the CRC and validation stages of Section VI, although rejection counts were not logged.

B. Reception Below the Noise Floor

In Set A the SNR was between −3.5 and −6.2 dB, so the signal power was below the noise power in the 125 kHz channel, yet the receiver remained in LINK OK. This is the ~21 dB CSS processing gain of Section III observed directly [4], [7]. The margin to the −7.5 dB limit was only 1.3–4.0 dB, so Set A was close to the edge of reliable demodulation. The −107 dBm RSSI lies about 16 dB above the −123 dBm sensitivity [15]; near sensitivity RSSI includes noise power, which is why SNR is the more meaningful margin. In Set B the SNR rose to +9.2 to +9.8 dB with a 31–40 dB stronger RSSI: a comfortable link.

C. Packet Loss Is Almost Independent of Signal Strength

If the loss were caused by the channel, an improvement of about 35 dB in RSSI and 13–16 dB in SNR should reduce it sharply [9]. Instead it fell only from 20.6–20.9 % to 20.0 % (Fig. 23). Unrounded, the Set B loss is 20.04–20.05 %, within 0.05 percentage points of exactly one packet in five. A reasonable reading is therefore:

- a **structural loss of 20 %**, independent of the channel and repeating every fifth packet; and
- a **small channel-related excess of 0.6–0.9 percentage points** in Set A, consistent with operating only 1.3–4.0 dB above the SNR limit.

This decomposition is an inference from seven snapshots, not a direct measurement.

D. Consistency of the Counters

Because LOST counts sequence gaps after the first accepted packet, PKT + LOST should trail the receiver SEQ by a constant offset set by the first accepted sequence number and any resynchronisation. In Set A the offset is 2 in all four observations, including the separately photographed Observation 4 (2248 + 594 = 2842 vs. SEQ 2844). In Set B it is 168 in all three (e.g., 1420 + 356 = 1776 vs. SEQ 1944), suggesting the receiver started or resynchronised about 168 packets into the transmitter's run. The constant offsets (Fig. 27) show that the counters are internally consistent and that the loss figures are not an artefact of a counter reset.

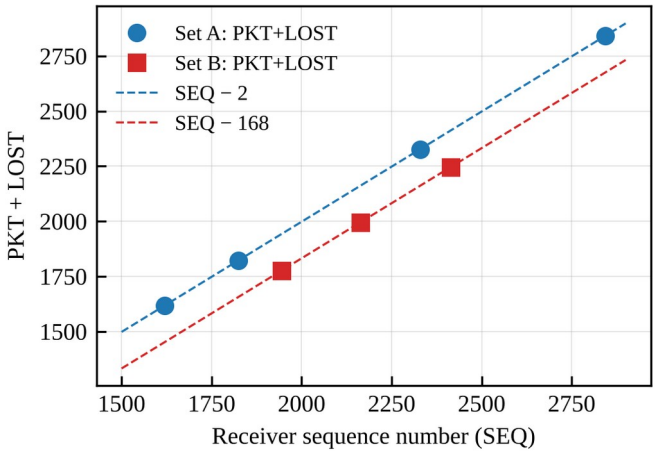


Fig. 27. PKT + LOST versus receiver SEQ. Each set lies on a line of constant offset (2 for Set A, 168 for Set B).

E. Acknowledgement Limitation

The transmitter requested an acknowledgement on every fifth command (`ACK_EVERY_N_PACKETS = 5`) and was designed to listen for it in an 80 ms window before sending the next command (Fig. 28). Across every reading the ACK count stayed at 0 while the receiver reported LINK OK. The forward link works; the return path never completed.

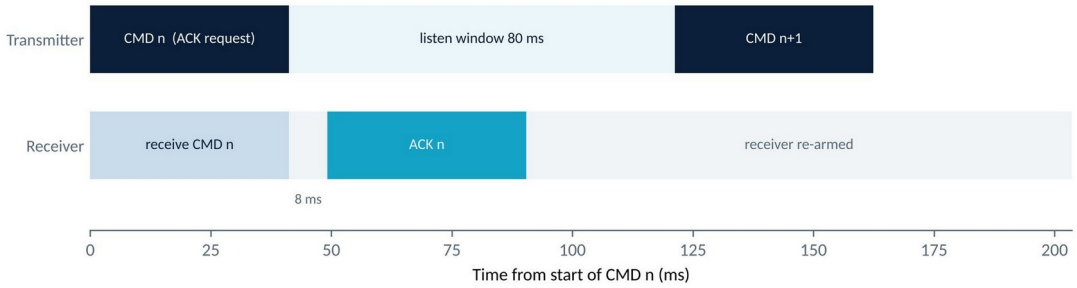


Fig. 28. Designed half-duplex acknowledgement schedule from firmware constants (41.2 ms airtime, 8 ms reply delay, 80 ms listen window). This is the intended timing; acknowledgement reception was not observed.

Working hypothesis (not yet confirmed): the 1-in-5 periodicity of the loss matches the 1-in-5 ACK request exactly. The simplest explanation consistent with both symptoms is that the packets carrying the ACK-request flag, or the packets immediately after the ACK exchange, are not being accepted by the receiver; if the request packets never arrive, no ACK is ever sent. Half-duplex turnaround and timing collisions of this kind are a known limit of LoRa links [10], [12].

Verification test: (1) set `ENABLE_ACK = false` and repeat Set B; if loss falls to near 0 %, the ACK exchange is confirmed as the cause. (2) With ACKs enabled, read the receiver's Serial loss lines; if the missing sequence numbers are every fifth one (4, 9, 14, …), the ACK-request packets are the ones being lost.

F. Effective Update Rate

From the firmware timing (calculated, not measured), each five-packet cycle contains four 50 ms intervals plus one ACK cycle of about 121 ms, about 321 ms in total, i.e. about 15.6 packets/s transmitted. With 20 % lost, about 12.5 valid commands per second reach the servos, a mean update interval of about 80 ms: well inside the 500 ms failsafe but below the 20 Hz design target. Fixing or disabling the ACK exchange would restore it. The channel occupancy is also high (41.2 ms of airtime per 50 ms, ~82 %), which matters in a shared ISM band [11], [12].

XI. Limitations and Troubleshooting

TABLE X
KNOWN LIMITATIONS

Limitation	Evidence	Impact	Recommended action
ACK path not working	ACK: 0, NO ACK/LINK LOST on every TX reading	No delivery confirmation	Run the verification test of Section X-E
~20 % structural loss	Table VIII, Fig. 23	≈12.5 Hz update instead of 20 Hz	As above; re-measure afterwards
Obs. 4 not simultaneous	RX SEQ 2844 > TX SEQ 2750	Cannot compare TX/RX values	Re-capture with both OLEDs in one frame
Test geometry not recorded	Sets defined only by RSSI	RSSI cannot be related to distance	Record separation and antenna placement
Servo response, failsafe not measured	No recorded test	Verified in code only	Unplug-TX test; scope the PWM pins
Design schematic out of date	Fig. 6: elevator on GPIO 26	Risk of wiring a servo onto DIO0	Use Figs. 3–4
Bench-scale test only	Both sets in one enclosure	Range unknown	Outdoor range test, SF7 vs. SF9
High channel occupancy	41.2 ms airtime per 50 ms	~82 % duty in shared ISM band	Check band rules before longer tests

TABLE XI
TROUBLESHOOTING GUIDE USED DURING BRING-UP

Symptom	Likely cause	Check
LORA INIT FAIL	SPI wiring or supply	NSS = 5, RST = 14, MISO/MOSI not swapped, 3.3 V at the module
OLED blank	I²C wiring or address	SDA = 21, SCL = 22, address 0x3C (run an I²C scanner)
Radio ready, no packets	Parameter mismatch / no antenna	Identical shared block in both sketches; antennas fitted; both 433 MHz
"Rejected: bad version"	Different protocol constants	Re-upload both sketches from the same source
Joystick not centred at 90°	Calibration	Read raw values on Serial; update `rawCenter` and deadband
Servo jitter or ESP32 resets	Servos on logic supply	Separate 5 V rail, common ground, bulk capacitor

XII. Future Work

Everything in this section is proposed, not implemented. In the near term: run the acknowledgement verification test and re-measure loss with ACKs disabled and enabled; measure the failsafe delay and servo pulse widths with an oscilloscope; log every packet (sequence number, RSSI, SNR, reject reason) over Serial; and carry out a documented range test at SF7 and SF9, plotting loss against distance and SNR [9].

The longer-term aim is to connect the link to a tuned flight controller (e.g., ArduPilot or INAV) and let the aircraft fly autonomously between two points, with LoRa serving as infrastructure-free communication after floods, cyclones or earthquakes have disabled other networks:

- **Standard receiver interface:** output CRSF or SBUS frames on an ESP32 UART instead of servo PWM so any flight controller reads the board as an RC receiver; MAVLink could carry commands and telemetry.
- **Mission upload:** send start and destination waypoints over LoRa; the flight controller handles take-off, navigation and landing with its own GPS, IMU, barometer and compass.
- **Range for data rate:** higher spreading factors extend range at the cost of update rate (Fig. 1), which suits mission upload and telemetry better than live stick control.
- **Failsafe and security:** continue the mission or return home on link loss; add working acknowledgements, frequency hopping and authentication or encryption of commands.

Autonomous navigation would still depend on GPS, and any flight would have to comply with the Drone Rules, 2021 [17] and the applicable 433 MHz band rules.

XIII. Conclusion

A one-way LoRa command link at 433 MHz was designed, implemented and evaluated using two ESP32 boards and SX1278 transceivers. The firmware frames three control commands into a CRC-protected 11-byte packet with a sequence number, transmits it at SF7 with a calculated 41.2 ms time on air, and validates every packet in seven stages before applying it. The measurements show that commands arrived intact (within 1°) and that the link held at negative SNR down to −6.2 dB, a direct demonstration of LoRa's spread-spectrum processing gain. Packet loss was 20.0–20.9 % and almost independent of a 35 dB change in signal strength; together with the complete absence of acknowledgements, this points to the acknowledgement exchange rather than the radio channel, a cause that remains to be confirmed by the proposed test. Servo actuation and the failsafe are implemented but not measured, and flight-controller integration remains future work. This is an educational, ground-based prototype and not a flight-certified control system.

Project video:
https://drive.google.com/drive/folders/17KGEKCCFoPgjZRtqW9QMCZfphlg0HTpz?usp=sharing

Acknowledgment

The author thanks Dr. B. Nagajayanthi, School of Electronics Engineering, Vellore Institute of Technology, Chennai, for guidance during the Digital Communication Systems (BECE306P) course and project reviews.

References

[1] U. Raza, P. Kulkarni, and M. Sooriyabandara, "Low power wide area networks: An overview," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 2, pp. 855–873, 2nd Quart. 2017.

[2] M. Centenaro, L. Vangelista, A. Zanella, and M. Zorzi, "Long-range communications in unlicensed bands: The rising stars in the IoT and smart city scenarios," *IEEE Wireless Commun.*, vol. 23, no. 5, pp. 60–67, Oct. 2016.

[3] Semtech Corporation, "LoRa modulation basics," Application Note AN1200.22, Rev. 2, May 2015.

[4] L. Vangelista, "Frequency shift chirp modulation: The LoRa modulation," *IEEE Signal Process. Lett.*, vol. 24, no. 12, pp. 1818–1821, Dec. 2017.

[5] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2001.

[6] S. Haykin, *Digital Communication Systems*. Hoboken, NJ, USA: Wiley, 2014.

[7] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, "A study of LoRa: Long range & low power networks for the Internet of Things," *Sensors*, vol. 16, no. 9, Art. no. 1466, Sep. 2016.

[8] Semtech Corporation, "LoRa modem designer's guide," Application Note AN1200.13, Rev. 1, Jul. 2013.

[9] J. Petäjäjärvi, K. Mikhaylov, A. Roivainen, T. Hänninen, and M. Pettissalo, "On the coverage of LPWANs: Range evaluation and channel attenuation model for LoRa technology," in *Proc. 14th Int. Conf. ITS Telecommun. (ITST)*, Copenhagen, Denmark, 2015, pp. 55–59.

[10] M. C. Bor, U. Roedig, T. Voigt, and J. M. Alonso, "Do LoRa low-power wide-area networks scale?" in *Proc. 19th ACM Int. Conf. Modeling, Anal. Simul. Wireless Mobile Syst. (MSWiM)*, Malta, 2016, pp. 59–67.

[11] O. Georgiou and U. Raza, "Low power wide area network analysis: Can LoRa scale?" *IEEE Wireless Commun. Lett.*, vol. 6, no. 2, pp. 162–165, Apr. 2017.

[12] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melià-Seguí, and T. Watteyne, "Understanding the limits of LoRaWAN," *IEEE Commun. Mag.*, vol. 55, no. 9, pp. 34–40, Sep. 2017.

[13] W. W. Peterson and D. T. Brown, "Cyclic codes for error detection," *Proc. IRE*, vol. 49, no. 1, pp. 228–235, Jan. 1961.

[14] Espressif Systems, "ESP32 series datasheet," Shanghai, China. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[15] Semtech Corporation, "SX1276/77/78/79 – 137 MHz to 1020 MHz low power long range transceiver," Datasheet, Rev. 7, May 2020.

[16] S. Mistry, "arduino-LoRa: An Arduino library for sending and receiving data using LoRa radios," GitHub repository. [Online]. Available: https://github.com/sandeepmistry/arduino-LoRa

[17] Ministry of Civil Aviation, Government of India, "The Drone Rules, 2021," *The Gazette of India: Extraordinary*, G.S.R. 589(E), Aug. 25, 2021.